COM7022 Machine Learning

Product Segmentation and Performance Clustering

Portfolio: Task 1 (Practical Project) and Task 2 (Critical Report)

Student ID: 25199053

Module: COM7022 Machine Learning, Assignment [5444]

Programme: MSc Data Science, Arden University Berlin

Dataset: Sales-[5444].csv (1,000 e-commerce fragrance listings)

GitHub repository (all materials): https://github.com/kvmmn/_COM7022_MachineLearning

Notebook: COM7022 - Assignment/COM7022_Clustering_25199053.ipynb

Word count: 2,892 words of body prose (Introduction 457, Task 1 1569, Task 2 866); captions, tables, code screenshots, references and appendix excluded

All figures, tables and code screenshots in this report are saved in the GitHub repository above. The caption under each one names the exact file.

1. Introduction

1.1 The company and the business context

I joined a fast-growing online marketplace as a Machine Learning Analyst. The company lets many independent sellers list the same kinds of products side by side, so its catalogue grows faster than any team can review by hand. Managers must still decide which listings to promote, which to restock and which to retire. They currently have no systematic way to tell a fast-selling staple from a slow premium item or a listing that is about to run out.

1.2 The data

The company supplied one file, Sales-[5444].csv, with **1,000 product listings and 10 columns**. The listings are men's fragrances. Each row holds the product name and description, a category label, the price, the stock shown to buyers, a text field with stock and units sold, the last update time and the item location. Each row is one seller's listing, not one unique product, so the clusters describe listings.

1.3 The problem

Three problems stand in the way of a data-driven catalogue. The catalogue is too large to review manually. The data is messy: values are missing, category names are inconsistent, and exact stock above 10 units is hidden. Finally, there are no labels that say which listing is a "best-seller" or a "slow mover", so a supervised model cannot be trained. The company needs a way to **discover** the natural groups in its own data.

1.4 Objectives

My own objectives for this project are:

- **O1. Prepare** the data so that every cleaning decision is evidence-based and reported.
- **O2. Group** listings by product characteristics and sales performance using K-Means, Hierarchical clustering and DBSCAN.
- **O3. Evaluate** the models with several different metrics, not one, and say what each metric can and cannot tell us.
- **O4. Choose** the model that is best technically and best for an e-commerce business.
- **O5. Translate** the segments into actions that non-technical managers can use.

1.5 Approach and method

I followed the standard machine learning workflow shown in Figure 1: understand the problem, audit and clean the data, transform and scale it, select features, reduce dimensions with PCA for viewing, fit three clustering techniques, evaluate them, choose one, profile the segments and turn them into business actions. All work is in Python (pandas, scikit-learn, SciPy, matplotlib) in one notebook that runs end to end on Google Colab with fixed random seeds. The notebook was re-run from scratch before writing this report, and it reproduced every number below.

The report has two parts. Task 1 documents the practical work: data preparation, model selection and evaluation. Task 2 is the critical report for managers: why K-Means was chosen, how the models performed, what the company should do and where the work could go next. Every figure, table and code screenshot names the file in the GitHub repository that produced it, so a reader can check any result.

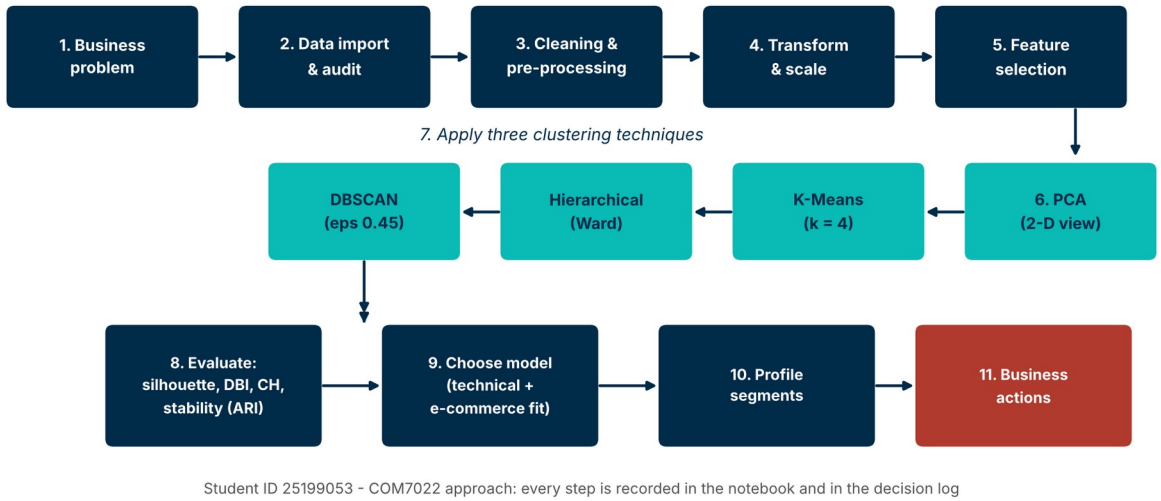


Figure 1. End-to-end workflow followed in this project. Source file: COM7022 - Assignment/output/figures/fig15_workflow.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig15_workflow.png

Task 1: Practical Project

2. Data preparation

2.1 Data audit: missing values and quality

I first inspected the file for missing values, duplicates, skew and dirty labels. There are no duplicate rows. Six columns have gaps, the largest being available (111 rows, 11.1%) and lastUpdated (53 rows, 5.3%) (Table 1 and Figure 2).

The audit exposed four issues that would mislead a clustering algorithm. **First**, price and units sold are strongly right-skewed: the most-sold listing has 54,052 sales against a median of about 50 (skewness 9.70). **Second**, the stock figure is **censored**: the marketplace shows “More than 10 available” and stores it as 10 (299 listings, including two lot-based listings). **Third**, 63 distinct category labels are really spelling and capitalisation variants of a handful of product types. **Fourth**, three prices are in Canadian dollars.

Column	Missing	Missing %
available	111	11.1
lastUpdated	53	5.3
sold	8	0.8
price	5	0.5
Catatory	3	0.3
text	3	0.3
Item_Name	1	0.1
priceWithCurrency	1	0.1

Table 1. Missing values per column in the raw file (1,000 rows). Source file: COM7022 - Assignment/output/tables/table01_missing_values.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table01_missing_values.csv

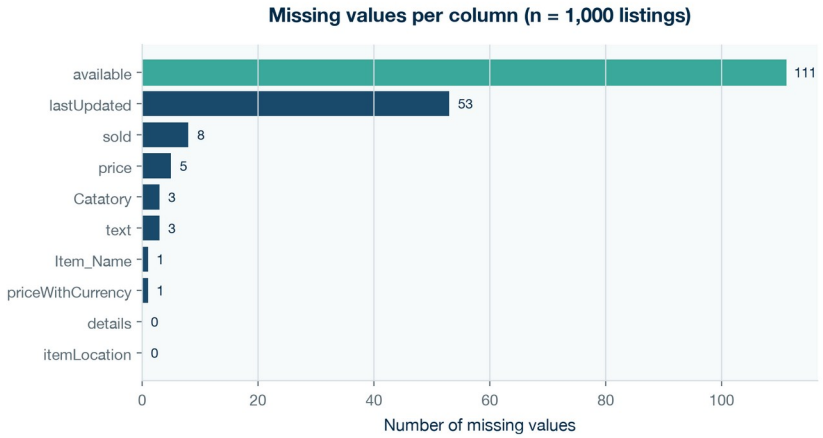


Figure 2. Missing values by column. Source file: COM7022 - Assignment/output/figures/fig01_missing_values.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig01_missing_values.png

2.2 Cleaning and pre-processing

My rule was to **recover** a missing value from another column of the same row before imputing anything, and to impute only where the reason for the gap is understood. Every step was written to a cleaning log (Table 2). In brief:

- **Price and units sold** were recovered from the text columns priceWithCurrency and text (5 prices, 2 sales figures), and the three Canadian-dollar prices were converted to US dollars.
- **Stock level** was rebuilt from the listing wording. “More than 10” and any larger number were capped at 11 so every row is on one scale, “Last One” became 1, and 62 “Limited quantity” rows received the median, with a flag column to mark them.
- **Product type**: the 63 category labels were merged into six clean types (Eau de Toilette, Eau de Parfum, Cologne, Parfum / Extrait, Generic fragrance, Other).
- **Unusable rows**: three listings with neither sales nor stock information were dropped, leaving **997 listings**.

Why recover instead of impute? Filling gaps with an average hides the reason a value is missing. Here the reason is known: the marketplace writes “Last One” or “Limited quantity” instead of a number, so the wording itself carries information. Recovering from the same row keeps each listing’s real values. The only imputation is the median for 62 stock values, flagged in a column so the effect could be checked. No listing was removed as an outlier, because the best-sellers are exactly the listings the business cares about.

Step	Rows affected	Note
Recover price from priceWithCurrency	5	Exact same value, only stored in text form
Convert CAD prices to USD	3	Fixed rate 0.73, documented approximation
Recover sold from text	2	Exact value from the listing wording
Set sold = 0 where text shows stock but no sales	3	Listing wording omits 'sold' when nothing has sold
Drop rows with no sales or stock information	3	Cannot be placed on the sales-performance axis
Stock: 'More than 10' coded as 11	299	Censored value, not exact
Stock: explicit values above 10 capped at 11	115	Same scale as the censored rows
Stock: 'Last One' coded as 1	47	Wording means one unit left
Stock: median imputation	62	Limited quantity / lot wording; flagged in stock_imputed
Collapse category labels into 6 product types	994	Case, spelling and wording variants
Missing brand set to 'Unbranded'	1	Single row
Log-transform price and sold	997	Outliers kept; log scale limits their pull on

		distances

Table 2. Cleaning log: every pre-processing step with the number of rows affected. Source file: COM7022 - Assignment/output/tables/table03_cleaning_log.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table03_cleaning_log.csv

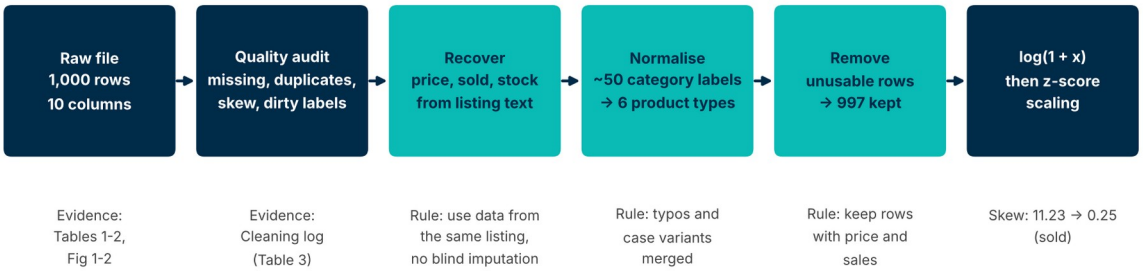


Figure 3. Data preparation flow from raw file to model-ready data. Source file: COM7022 - Assignment/output/figures/fig16_data_prep_flow.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig16_data_prep_flow.png

The screenshots below show the code behind these steps (the full code is in the notebook and in Appendix A).

COM7022_Clustering_25199053.ipynb | Data import

```
DATA_FILE = "Sales-[5444].csv"
candidate_paths = [Path(DATA_FILE), Path("..") / DATA_FILE, Path("/content") / DATA_FILE]

data_path = None
for path in candidate_paths:
    if path.exists():
        data_path = path
        break

if data_path is None:
    try:
        from google.colab import files  # only available on Colab
        print("CSV not found. Please upload", DATA_FILE)
        uploaded = files.upload()
        data_path = Path(list(uploaded.keys())[0])
    except ImportError:
        raise FileNotFoundError("Place " + DATA_FILE + " next to this notebook.")

raw = pd.read_csv(data_path)
print("Loaded:", data_path)
print("Shape:", raw.shape[0], "rows x", raw.shape[1], "columns")
raw.head()
```

Figure 4. Code screenshot: importing the data Source file: COM7022 - Assignment/output/screenshots/code01_import_data.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code01_import_data.png

COM7022_Clustering_25199053.ipynb | Missing-value audit

```python
# Missing values per column, as counts and percentages
missing = pd.DataFrame({
    "missing_count": raw.isna().sum(),
    "missing_pct": (raw.isna().mean() * 100).round(1),
})
missing = missing.sort_values("missing_count", ascending=False)
print("Full-row duplicates:", raw.duplicated().sum())
print("Duplicate listing descriptions:", raw["details"].duplicated().sum())
save_table(missing, "table01_missing_values.csv")
missing
```

Figure 5. Code screenshot: missing-value audit Source file: COM7022 - Assignment/output/screenshots/code02_missing_values.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code02_missing_values.png

COM7022_Clustering_25199053.ipynb | Cleaning: recover price

```python
df = raw.copy()
cleaning_log = []


def log_step(step, rows_affected, note):
    cleaning_log.append({"step": step, "rows_affected": int(rows_affected), "note": note})


# 3.1 Price: recover missing prices from priceWithCurrency (same listing, same number)
price_from_text = df["priceWithCurrency"].str.extract(r"\$([\d,]+\.?\d*)")[0]
price_from_text = price_from_text.str.replace(",", "").astype(float)
recovered = df["price"].isna() & price_from_text.notna()
df.loc[recovered, "price"] = price_from_text[recovered]
log_step("Recover price from priceWithCurrency", recovered.sum(), "Exact same value, only stored in text form")

# Check: where both exist, do they agree?
both = df["price"].notna() & price_from_text.notna()
print("Price disagreements where both exist:", (abs(df.loc[both, "price"] - price_from_text[both]) > 0.01).sum())

# 3.2 Currency: convert the three Canadian-dollar prices to USD
# Approximate May 2024 rate (1 CAD ~ 0.73 USD). Only 3 rows, so the exact rate barely matters.
CAD_TO_USD = 0.73
is_cad = df["priceWithCurrency"].fillna("").str.startswith("C")
df.loc[is_cad, "price"] = (df.loc[is_cad, "price"] * CAD_TO_USD).round(2)
log_step("Convert CAD prices to USD", is_cad.sum(), "Fixed rate 0.73, documented approximation")
```

Figure 6. Code screenshot: recovering price Source file: COM7022 - Assignment/output/screenshots/code03_price_recovery.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code03_price_recovery.png

COM7022_Clustering_25199053.ipynb | Cleaning: recover units sold

```python
# 3.3 Units sold: recover from the 'text' column ("... / 116 sold")
sold_from_text = df["text"].str.extract(r"([\d,]+) sold")[0].str.replace(",", "").astype(float)
recovered = df["sold"].isna() & sold_from_text.notna()
df.loc[recovered, "sold"] = sold_from_text[recovered]
log_step("Recover sold from text", recovered.sum(), "Exact value from the listing wording")

# Check the two sources agree everywhere else
both = df["sold"].notna() & sold_from_text.notna()
print("Sold disagreements where both exist:", (df.loc[both, "sold"] != sold_from_text[both]).sum())

# Listings whose text shows stock but no 'sold' part have not sold yet, so sold = 0
no_sales_yet = df["sold"].isna() & df["text"].notna() & ~df["text"].str.contains("sold", na=False)
df.loc[no_sales_yet, "sold"] = 0
log_step("Set sold = 0 where text shows stock but no sales", no_sales_yet.sum(), "Listing wording omits 'sold' when nothing has sold")

# Rows with no text and no sold value carry no sales or stock information at all
no_info = df["sold"].isna() & df["text"].isna()
print(df.loc[no_info, ["Item_Name", "price", "available", "text", "sold"]])
df = df[~no_info].copy()
log_step("Drop rows with no sales or stock information", no_info.sum(), "Cannot be placed on the sales-performance axis")
```

Figure 7. Code screenshot: recovering units sold Source file: COM7022 - Assignment/output/screenshots/code04_sold_recovery.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code04_sold_recovery.png

COM7022_Clustering_25199053.ipynb | Cleaning: stock level and censoring

```python
# 3.4 Stock level: rebuild it from the listing wording so every row has a value.
# The marketplace hides exact numbers above 10 ("More than 10 available"), so I cap all
# stock values at 11, meaning "more than 10". This keeps the measure honest and comparable.
text = df["text"].fillna("")
stock = text.str.extract(r"^(\d+) available")[0].astype(float)
stock[text.str.startswith("More than")] = 11
stock[text.str.startswith("Last One")] = 1
capped_above_10 = (stock > 11).sum()
stock = stock.clip(upper=11)

# "Limited quantity available" and lot listings give no number: impute the median, but keep a flag
stock_imputed = stock.isna()
df["stock_imputed"] = stock_imputed.astype(int)
stock = stock.fillna(stock.median())
df["stock_level"] = stock

log_step("Stock: 'More than 10' coded as 11", text.str.startswith("More than").sum(), "Censored value, not exact")
log_step("Stock: explicit values above 10 capped at 11", capped_above_10, "Same scale as the censored rows")
log_step("Stock: 'Last One' coded as 1", text.str.startswith("Last One").sum(), "Wording means one unit left")
log_step("Stock: median imputation", stock_imputed.sum(), "Limited quantity / lot wording; flagged in stock_imputed")
print(df["stock_level"].value_counts().sort_index())
```

Figure 8. Code screenshot: rebuilding stock level and handling censoring Source file: COM7022 - Assignment/output/screenshots/code05_stock_level.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code05_stock_level.png

COM7022_Clustering_25199053.ipynb | Cleaning: normalise product type

```python
# 3.5 Product type: collapse the messy category labels into six groups


def group_category(label):
    if pd.isna(label):
        return "Other"
    s = str(label).strip().lower()
    if s in ["", "/", "y", "does not apply", "unknown", "various", "assorted"]:
        return "Other"
    if "," in s or "/" in s:
        return "Other"                          # listings that name several types at once
    if "toil" in s or s == "edt":
        return "Eau de Toilette"                # includes the misspelling 'toillette'
    if "eau de parfum" in s or "eau de perfume" in s:
        return "Eau de Parfum"
    if "parfum" in s or "extrait" in s or "elixir" in s:
        return "Parfum / Extrait"
    if "cologne" in s or s == "edc":
        return "Cologne"
    if "perfume" in s or "fragrance" in s:
        return "Generic fragrance"
    return "Other"                              # body sprays, oils, deodorants, gift sets


df["product_type"] = df["Catatory"].apply(group_category)
print("Raw labels:", df["Catatory"].nunique(), "-> groups:", df["product_type"].nunique())
print(df["product_type"].value_counts())
log_step("Collapse category labels into 6 product types", df["Catatory"].notna().sum(), "Case, spelling and wording variants")

# 3.6 Brand: tidy whitespace; a missing brand is treated as 'Unbranded'
df["brand"] = df["Item_Name"].fillna("Unbranded").str.strip()
log_step("Missing brand set to 'Unbranded'", df["Item_Name"].isna().sum(), "Single row")

# 3.7 Location: keep the country (last part of 'City, State, Country')
df["country"] = df["itemLocation"].str.split(",").str[-1].str.strip()
df["ships_from_us"] = (df["country"] == "United States").astype(int)

# 3.8 Last update: parse dates (two month formats) and turn them into 'days since update'
dates = pd.to_datetime(df["lastUpdated"].str.replace(r" P[DS]T", "", regex=True), format="mixed", errors="coerce")
df["days_since_update"] = (dates.max() - dates).dt.total_seconds() / 86400
print("Unparsed or missing dates:", dates.isna().sum(), "| latest update:", dates.max())

# 3.9 Bottle size in ounces, parsed from the description (used for profiling, not clustering)
size = df["details"].str.extract(r"(\d+(?:\.\d+)?)\s*(?:fl\.?\s*)?oz", flags=re.IGNORECASE)[0].astype(float)
df["size_oz"] = size.where(size.between(0.3, 8.5))   # ignore implausible parses
print("Bottle size found for", df["size_oz"].notna().sum(), "listings")
```

Figure 9. Code screenshot: normalising product type Source file: COM7022 - Assignment/output/screenshots/code06_product_type.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code06_product_type.png

2.3 Transformation, scaling and categorical variables

Log transform. Distance-based clustering is dominated by extreme values, and sales are extremely skewed. I applied log(1 + x) to price and units sold, which keeps the best-sellers in the data but stops them from dominating the distances (Figure 10). After cleaning, the skewness of units sold falls from 9.72 to 0.52 and of price from 1.97 to 0.09 (Table 3).

Scaling. I used z-score standardisation so that each feature has mean 0 and standard deviation 1. Min-max scaling was tested as a sensitivity check and changed the grouping substantially (agreement 0.49, Section 4.4), because it exaggerates the narrow 1 to 11 stock scale.

Categorical variables. Brand has 246 distinct values, so one-hot encoding would add hundreds of columns and let a single brand drive the distances. Product type, brand and country were therefore kept out of the distance calculation and used afterwards to describe the clusters.

These choices were tested rather than assumed. In the sensitivity checks (Section 4.4) K-Means gave almost the same groups with a different seed and with the top 1% of sellers removed, so the pre-processing makes the result robust. It only failed with a different scaling method, which is why scaling is documented as a deliberate decision.

Variable	count	mean	std	min	median	max	skewness
price	997.0	46.07	34.75	3.0	35.62	259.09	1.97
sold	997.0	771.63	3203.77	0.0	49.0	54052.0	9.72
stock_level	997.0	7.73	3.53	1.0	9.0	11.0	-0.57
log_price	997.0	3.63	0.67	1.39	3.6	5.56	0.09
log_sold	997.0	4.29	2.08	0.0	3.91	10.9	0.52

Table 3. Descriptive statistics after cleaning (997 listings), including skewness before and after the log transform. Source file: COM7022 - Assignment/output/tables/table04_descriptive_statistics_clean.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table04_descriptive_statistics_clean.csv

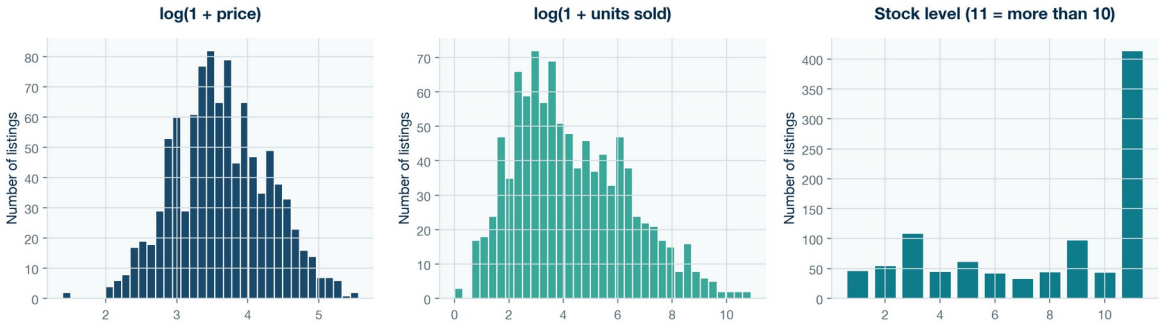


Figure 10. Distributions after the log transform. Source file: COM7022 - Assignment/output/figures/fig03_transformed_distributions.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig03_transformed_distributions.png

COM7022_Clustering_25199053.ipynb | Transformation: log(1 + x)

```
# 3.10 Transform the skewed measures. log(1 + x) keeps zero sales valid (log of 0 is undefined).
df["log_price"] = np.log1p(df["price"])
df["log_sold"] = np.log1p(df["sold"])

skew_table = pd.DataFrame({
    "skew_before": [skew(df["price"]), skew(df["sold"])],
    "skew_after_log": [skew(df["log_price"]), skew(df["log_sold"])],
}, index=["price", "sold"]).round(2)
print(skew_table)

# Outliers: count them with the 1.5 x IQR rule on the raw scale, but keep them.
# Best-selling listings are exactly what the business wants to find, so removing them would be wrong.
q1, q3 = df["sold"].quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)
print("Listings above the sold upper fence (" + str(round(upper_fence)) + "):", (df["sold"] > upper_fence).sum())
log_step("Log-transform price and sold", len(df), "Outliers kept; log scale limits their pull on distances")

cleaning_table = pd.DataFrame(cleaning_log)
save_table(cleaning_table, "table03_cleaning_log.csv")
print("Rows after cleaning:", len(df))
cleaning_table
```

Figure 11. Code screenshot: log transform Source file: COM7022 - Assignment/output/screenshots/code07_log_transform.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code07_log_transform.png

COM7022_Clustering_25199053.ipynb | Scaling: StandardScal

```
FEATURES = ["log_price", "log_sold", "stock_level"]
scaler = StandardScaler()
X = scaler.fit_transform(df[FEATURES])
print("Feature matrix:", X.shape)
print("Means after scaling:", X.mean(axis=0).round(3))
print("Std after scaling:", X.std(axis=0).round(3))
```

Figure 12. Code screenshot: standard scaling of the selected features Source file: COM7022 - Assignment/output/screenshots/code08_scaling.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code08_scaling.png

2.4 Feature selection

The brief asks for groups based on product characteristics and sales performance, so I screened candidate features with a Spearman correlation matrix (Figure 13) and compared five feature sets by silhouette score (Table 4). I selected **log price, log units sold and stock level** (Set B). They give three distinct business views: market position, sales performance and availability. Set A (price and sales only) scores slightly higher but ignores availability, which the business needs to spot stock-outs. Adding days since update or bottle size lowered the scores and added little meaning.

Feature set	Features	k=2	k=3	k=4	k=5	k=6
A: price + sold	2	0.351	0.377	0.356	0.351	0.354
B: price + sold + stock	3	0.337	0.297	0.307	0.308	0.297
C: B + days since update	4	0.31	0.315	0.277	0.288	0.284
D: B + bottle size	4	0.274	0.308	0.271	0.272	0.272
E: B + days + size	5	0.256	0.259	0.292	0.255	0.258

Table 4. Silhouette score of K-Means for five feature sets (higher is better). Source file: COM7022 - Assignment/output/tables/table05_feature_set_scan.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table05_feature_set_scan.csv

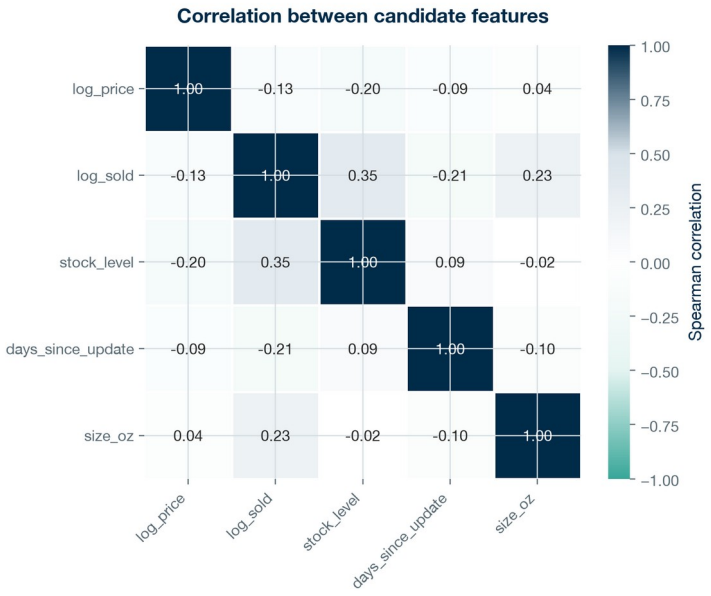


Figure 13. Spearman correlation between candidate features. Source file: COM7022 - Assignment/output/figures/fig04_feature_correlation.png |

https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig04_feature_correlation.png

3. Model selection

3.1 What is clustering?

Clustering is **unsupervised learning**: the algorithm finds groups of similar items without being told the right answer. Listings in the same cluster are similar to each other and different from listings in other clusters. That fits this problem because the company has no labels for its listings.

3.2 The three techniques

Technique	How it works	Strengths	Weaknesses
K-Means	Places k centres and assigns each listing to the nearest one, then moves the centres until stable.	Fast, simple, every listing gets a group, easy to explain.	Needs k in advance; assumes round, similar-sized groups.
Hierarchical (Ward)	Starts with every listing alone and merges the most similar pair step by step; the result is a tree (dendrogram).	No need to fix k; the tree shows structure.	Merges are final; slow on large data; sensitive to sampling.
DBSCAN	Grows groups from dense regions and labels isolated listings as noise.	Finds odd shapes; flags outliers; no k needed.	Needs a dense, well-separated structure and careful settings.

Table 5. The three clustering techniques compared.

3.3 Dimensionality reduction with PCA

With three features PCA is not needed to remove noise, but I used it to draw all clusters on one two-dimensional map. The first two components keep **78.7%** of the variance (PC1 48.6%, PC2 30.1%). PC1 behaves like commercial traction (sales and stock load positively, price negatively) and PC2 like price tier (loading 0.91 on log price), so the map has a business meaning (Figure 14).

PCA also served as a check on the feature choice. All three features load strongly on the first two components, so none is redundant, and the third component (21.3% of variance) mainly contrasts sales with stock. It adds detail but is not needed for viewing the groups.



Figure 14. PCA explained variance (left) and feature loadings (right). Source file: COM7022 - Assignment/output/figures/fig05_pca_variance_biplot.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig05_pca_variance_biplot.png

COM7022_Clustering_25199053.ipynb | PCA

```
pca = PCA(random_state=SEED)
X_pca_full = pca.fit_transform(X)
explained = pca.explained_variance_ratio_
loadings = pd.DataFrame(pca.components_.T, index=FEATURES,
                        columns=["PC" + str(i + 1) for i in range(len(FEATURES))]).round(3)
print("Explained variance ratio:", explained.round(3))
print("Cumulative:", explained.cumsum().round(3))
save_table(loadings, "table06_pca_loadings.csv")
loadings
```

Figure 15. Code screenshot: PCA Source file: COM7022 - Assignment/output/screenshots/code09_pca.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code09_pca.png

4. Implementation and evaluation

4.1 K-Means

I scanned k from 2 to 10 using the elbow in inertia and three indices (Table 6). The silhouette is highest at k = 2 (0.337), but two groups only separate "sells" from "does not sell". At **k = 4** inertia has dropped 39% from k = 2, the Davies-Bouldin index is the lowest of all (1.015), and every cluster still holds at least 22% of the listings.

K-Means used k-means++ starting points, 20 restarts per fit and a fixed seed (42), so the answer is repeatable. The map in Figure 17 shows four regions arranged along traction (PC1) and price (PC2), with soft borders where neighbouring groups meet. That is what a continuum looks like, and it explains why the silhouette is moderate rather than high.

k	Inertia	Silhouette	Davies-Bouldin	Calinski-Harabasz	Smallest cluster
2	1943.0	0.337	1.211	536.7	393
3	1501.0	0.297	1.228	493.4	298
4	1186.7	0.307	1.015	503.3	219
5	974.4	0.308	1.018	513.3	156
6	862.5	0.297	1.053	489.1	141
7	766.7	0.307	1.043	478.7	84
8	691.8	0.296	1.057	469.6	82
9	640.9	0.296	1.029	452.9	59
10	594.6	0.294	1.037	442.0	53

Table 6. K-Means scan: inertia and validation indices for k = 2 to 10. Source file: COM7022 - Assignment/output/tables/table07_kmeans_k_scan.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table07_kmeans_k_scan.csv

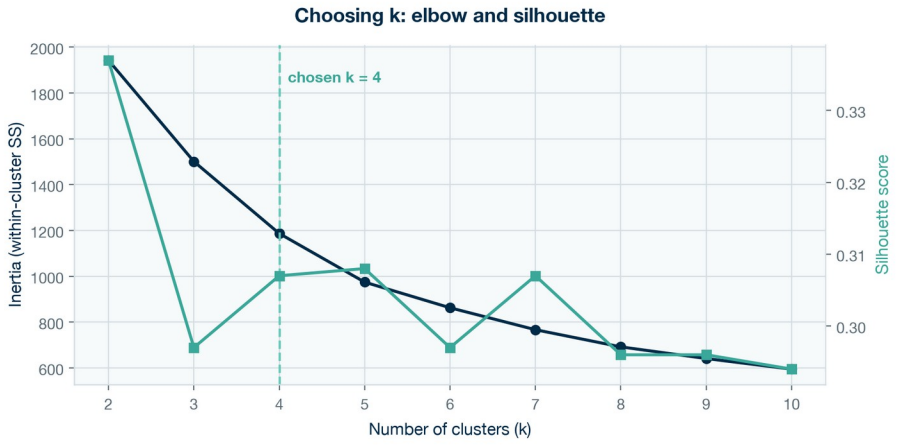


Figure 16. Elbow (inertia) and silhouette against k. Source file: COM7022 - Assignment/output/figures/fig06_kmeans_elbow_silhouette.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig06_kmeans_elbow_silhouette.png

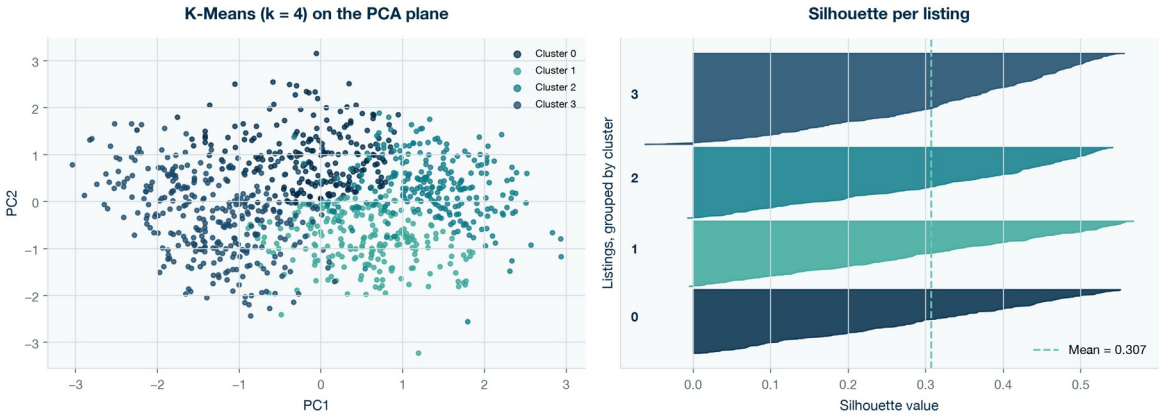


Figure 17. K-Means clusters on the PCA map (left) and silhouette profile (right). Source file: COM7022 - Assignment/output/figures/fig07_kmeans_clusters_silhouette.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig07_kmeans_clusters_silhouette.png

COM7022_Clustering_25199053.ipynb | K-Means: scan k

```
k_values = range(2, 11)
kmeans_scan = []
for k in k_values:
    model = KMeans(n_clusters=k, n_init=20, random_state=SEED)
    labels_k = model.fit_predict(X)
    kmeans_scan.append({
        "k": k,
        "inertia": round(model.inertia_, 1),
        "silhouette": round(silhouette_score(X, labels_k), 3),
        "davies_bouldin": round(davies_bouldin_score(X, labels_k), 3),
        "calinski_harabasz": round(calinski_harabasz_score(X, labels_k), 1),
        "smallest_cluster": int(np.bincount(labels_k).min()),
    })
kmeans_scan = pd.DataFrame(kmeans_scan).set_index("k")
save_table(kmeans_scan, "table07_kmeans_k_scan.csv")
kmeans_scan
```

Figure 18. Code screenshot: scanning k for K-Means Source file: COM7022 - Assignment/output/screenshots/code10_kmeans_scan.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code10_kmeans_scan.png

4.2 Hierarchical clustering

Following the taught approach I compared four linkages on the same distances (Table 7). Single and average linkage put 994 and 989 listings into one cluster (“chaining”), and complete linkage gave very uneven groups. **Ward** linkage gave balanced groups (358, 237, 235 and 167 listings) and the best silhouette (0.253), so I carried it forward. Its grouping agrees only moderately with K-Means (Adjusted Rand Index 0.41).

The dendrogram (Figure 19) splits into four branches at a merge distance of about 21, so a four-cluster cut is reasonable and matches K-Means, which lets the two methods be compared on equal terms. The branches are still unstable under resampling, as Section 4.4 shows.

Linkage	Cophenetic correlation	Silhouette at k = 4	Cluster sizes
single	0.362	0.217	994/1/1/1
complete	0.536	0.197	424/388/180/5
average	0.631	0.206	989/6/1/1
ward	0.579	0.253	358/237/235/167

Table 7. Four linkage methods compared on the same data. Source file: COM7022 - Assignment/output/tables/table08_linkage_comparison.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table08_linkage_comparison.csv

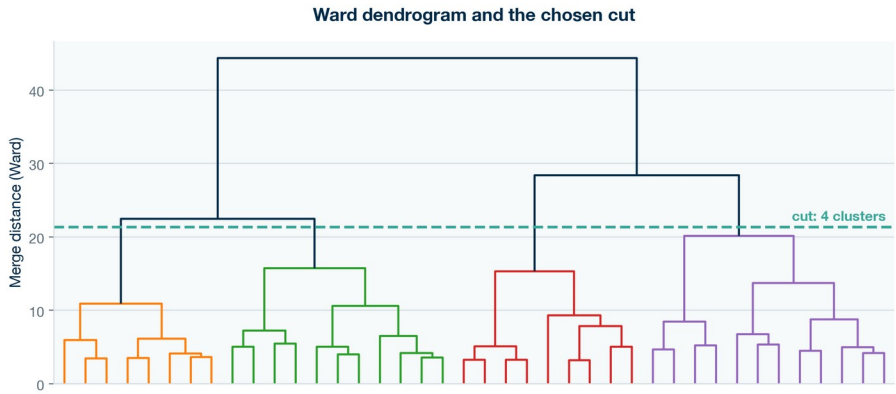


Figure 19. Ward dendrogram cut at four clusters. Source file: COM7022 - Assignment/output/figures/fig09_ward_dendrogram_cut.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig09_ward_dendrogram_cut.png

COM7022_Clustering_25199053.ipynb | Hierarchical: four linkages

```
linkage_methods = ["single", "complete", "average", "ward"]
distances = pdist(X)
linkage_results = {}
linkage_rows = []
for method in linkage_methods:
    Z = linkage(X, method=method)
    linkage_results[method] = Z
    coph_corr, _ = cophenet(Z, distances)
    labels_h = fcluster(Z, t=K_FINAL, criterion="maxclust")
    sizes = np.bincount(labels_h)[1:]
    linkage_rows.append({
        "linkage": method,
        "cophenetic_corr": round(coph_corr, 3),
        "silhouette_at_k": round(silhouette_score(X, labels_h), 3) if len(set(labels_h)) > 1 else np.nan,
        "cluster_sizes": "/".join(str(s) for s in sorted(sizes, reverse=True)),
    })
linkage_table = pd.DataFrame(linkage_rows).set_index("linkage")
save_table(linkage_table, "table08_linkage_comparison.csv")
linkage_table
```

Figure 20. Code screenshot: four linkages Source file: COM7022 - Assignment/output/screenshots/code11_hierarchical.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code11_hierarchical.png

4.3 DBSCAN

I set min_samples to 6 (twice the number of features) and read eps from the k-distance curve, then tested 32 combinations (Figure 21, Figure 22; the full grid is saved in output/tables/table10_dbscan_grid.csv). Small eps values (0.35 or below) split the data into many tiny groups and call 23% to 89% of listings noise; larger values merge everything. The best compromise, **eps = 0.45**, still puts 904 listings (90.7%) in one cluster, 4 in another and 89 (8.9%) in noise. The silhouette on clustered points is slightly negative (-0.014). The k-distance curve has no sharp knee because listings form a continuum, not separate dense islands. DBSCAN is therefore a poor segmenter here, but its noise points are useful: they are cheaper (median $26 against $36), sell more (median 163 against about 46 units) and carry less stock (5 against 9).

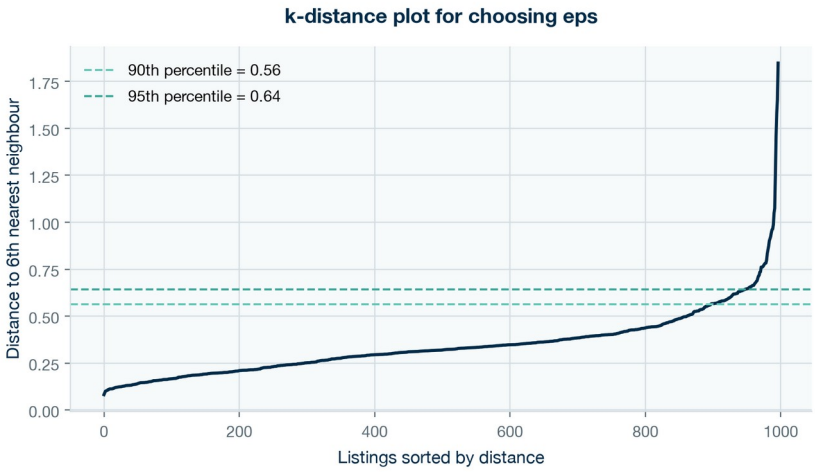


Figure 21. Sorted k-distance curve used to choose eps. Source file: COM7022 - Assignment/output/figures/fig10_dbscan_k_distance.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig10_dbscan_k_distance.png

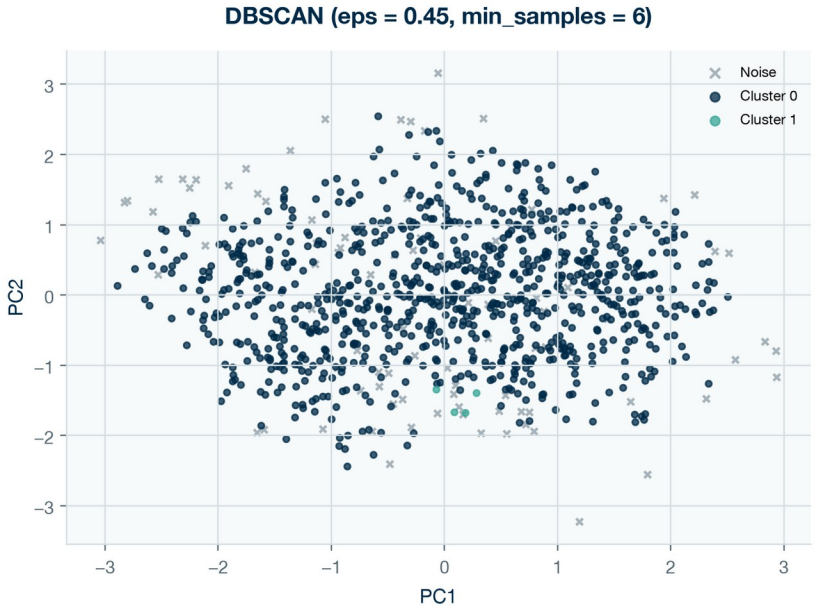


Figure 22. DBSCAN result on the PCA map; grey points are noise. Source file: COM7022 - Assignment/output/figures/fig11_dbscan_clusters.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig11_dbscan_clusters.png

COM7022_Clustering_25199053.ipynb | DBSCAN: k-distance

```
MIN_SAMPLES = 2 * len(FEATURES)
neighbours = NearestNeighbors(n_neighbors=MIN_SAMPLES).fit(X)
dist_k, _ = neighbours.kneighbors(X)
k_distance = np.sort(dist_k[:, -1])

# Figure 10: k-distance plot used to pick eps
fig, ax = plt.subplots(figsize=(8.5, 4.2))
ax.plot(k_distance, color=MIDNIGHT, linewidth=2)
for q, colour in [(0.90, TIFFANY), (0.95, TIFFANY_DEEP)]:
    ax.axhline(np.quantile(k_distance, q), color=colour, linestyle="--", linewidth=1.3,
               label=str(int(q * 100)) + "th percentile = " + str(round(np.quantile(k_distance, q), 2)))
ax.set_xlabel("Listings sorted by distance")
ax.set_ylabel("Distance to " + str(MIN_SAMPLES) + "th nearest neighbour")
ax.set_title("k-distance plot for choosing eps")
ax.legend(loc="upper left")
save_figure(fig, "fig10_dbscan_k_distance.png")
plt.show()
```

Figure 23. Code screenshot: DBSCAN k-distance Source file: COM7022 - Assignment/output/screenshots/code12_dbscan_kdist.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code12_dbscan_kdist.png

4.4 Evaluation with several metrics

Accuracy cannot be used because there are no true labels. I therefore combined five kinds of evidence, each answering a different question (Table 8).

Metric	What it measures	Better when	Limitation
Silhouette	How much closer a listing is to its own cluster than to the next one (-1 to 1).	Higher	Says nothing about stability or usefulness; penalises DBSCAN's uneven shapes.
Davies-Bouldin	Average similarity between each cluster and its closest rival (compactness vs separation).	Lower	Favours round clusters.
Calinski-Harabasz	Ratio of between-cluster to within-cluster spread.	Higher	Rises with convex clusters; unreliable for DBSCAN.
Stability (ARI)	Agreement of labels when 20% of listings are dropped (30 repeats). 1 = identical.	Higher	Shows whether the groups are real or an accident of the sample.
Sensitivity	Agreement after changing seed, scaling method or removing the top 1% of sellers.	Higher	Tests robustness to my own choices.
Noise % and cluster sizes	Share of listings left unassigned and the smallest and largest cluster.	Low noise, balanced	Business metric: can every listing be acted on?

Table 8. Evaluation metrics: what each one measures and what it cannot tell us.

Model	Clusters	Noise %	Silhouette	Davies-Bouldin	Calinski-Harabasz	Stability ARI	Smallest %	Largest %
K-Means	4	0.0	0.307	1.015	503.3	0.932	22.0	31.3
Hierarchical (Ward)	4	0.0	0.253	1.257	402.9	0.481	16.8	35.9
DBSCAN	2	8.9	-0.014	1.172	3.5	0.749	0.4	90.7

Table 9. Model comparison on all metrics (K-Means is best or equal-best on every column except cluster-size balance). Source file: COM7022 - Assignment/output/tables/table11_model_comparison.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table11_model_comparison.csv

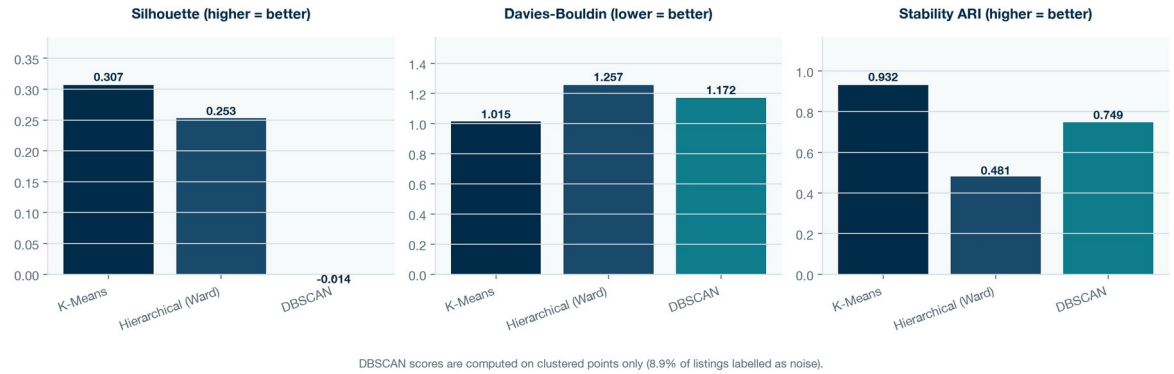


Figure 24. Side-by-side comparison of the three models. Source file: COM7022 - Assignment/output/figures/fig12_model_comparison.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig12_model_comparison.png

K-Means wins on every internal index (silhouette 0.307, Davies-Bouldin 1.015, Calinski-Harabasz 503) and on stability (ARI 0.93 ± 0.03 against 0.48 for Ward). It is also robust to my own choices: different seeds give ARI 0.98, removing the top 1% of sellers gives 0.94, but switching to min-max scaling gives only 0.49, which is why I standardised. Ward's tree is unstable because early merges cannot be undone.

How to read these numbers. A silhouette of 0.31 is commonly read as moderate structure: listings are on average closer to their own group than to another, but the groups touch. This agrees with the continuous shape seen in the PCA map. The Davies-Bouldin and Calinski-Harabasz indices rank the models in the same order as the silhouette, which gives more confidence than any single index. Stability adds something the indices cannot: a model can score well and still change when the sample changes, as Ward does. DBSCAN's scores are not directly comparable because they ignore the 8.9% of listings it calls noise.

Sensitivity check	ARI
K-Means across 10 seeds (mean ARI)	0.984
z-score vs min-max scaling (ARI)	0.492
with vs without top 1% sellers (ARI)	0.937

Table 10. Sensitivity checks on the K-Means solution. Source file: COM7022 - Assignment/output/tables/table12_sensitivity_checks.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table12_sensitivity_checks.csv

COM7022_Clustering_25199053.ipynb | Evaluation metrics function

```python
def internal_scores(labels, ignore_noise=False):
    labels = np.asarray(labels)
    mask = np.ones(len(labels), dtype=bool)
    if ignore_noise:
        mask = labels != -1
    return {
        "silhouette": round(silhouette_score(X[mask], labels[mask]), 3),
        "davies_bouldin": round(davies_bouldin_score(X[mask], labels[mask]), 3),
        "calinski_harabasz": round(calinski_harabasz_score(X[mask], labels[mask]), 1),
    }


def bootstrap_stability(fit_function, n_runs=30, frac=0.8):
    # Fit on the full data once, then on random subsamples, and compare labels on the shared rows
    full_labels = fit_function(X)
    rng = np.random.default_rng(SEED)
    scores = []
    for run in range(n_runs):
        idx = rng.choice(len(X), size=int(frac * len(X)), replace=False)
        sub_labels = fit_function(X[idx])
        scores.append(adjusted_rand_score(full_labels[idx], sub_labels))
    return np.mean(scores), np.std(scores)


def fit_kmeans(data):
    return KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(data)


def fit_ward(data):
    return AgglomerativeClustering(n_clusters=K_FINAL, linkage="ward").fit_predict(data)


def fit_dbscan(data):
    return DBSCAN(eps=EPS_FINAL, min_samples=MIN_SAMPLES).fit_predict(data)


comparison_rows = []
for name, column, fit_function, ignore_noise in [
        ("K-Means", "kmeans", fit_kmeans, False),
        ("Hierarchical (Ward)", "ward", fit_ward, False),
        ("DBSCAN", "dbscan", fit_dbscan, True)]:
    labels = df[column].values
    scores = internal_scores(labels, ignore_noise=ignore_noise)
    stab_mean, stab_std = bootstrap_stability(fit_function)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    sizes = pd.Series(labels[labels != -1]).value_counts()
    row = {"model": name, "clusters": n_clusters, "noise_pct": round((labels == -1).mean() * 100, 1)}
    row.update(scores)
    row["stability_ARI_mean"] = round(stab_mean, 3)
    row["stability_ARI_sd"] = round(stab_std, 3)
    row["smallest_cluster_pct"] = round(sizes.min() / len(labels) * 100, 1)
    row["largest_cluster_pct"] = round(sizes.max() / len(labels) * 100, 1)
    comparison_rows.append(row)

comparison = pd.DataFrame(comparison_rows).set_index("model")
save_table(comparison, "table11_model_comparison.csv")
comparison
```

Figure 25. Code screenshot: evaluation metrics function Source file: COM7022 - Assignment/output/screenshots/code13_metrics.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code13_metrics.png

COM7022_Clustering_25199053.ipynb | Stability (ARI) across seeds

```
# Seed stability for K-Means: does the answer change with the random start?
seed_labels = [KMeans(n_clusters=K_FINAL, n_init=20, random_state=s).fit_predict(X) for s in range(10)]
seed_ari = [adjusted_rand_score(seed_labels[0], lab) for lab in seed_labels[1:]]
print("K-Means ARI across 10 seeds: min", round(min(seed_ari), 3), "mean", round(np.mean(seed_ari), 3))

# Sensitivity 1: min-max scaling instead of z-scores (as used in the lessons)
X_minmax = MinMaxScaler().fit_transform(df[FEATURES])
km_minmax = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(X_minmax)
print("ARI z-score vs min-max K-Means:", round(adjusted_rand_score(df["kmeans"], km_minmax), 3))

# Sensitivity 2: drop the top 1% sellers and re-fit; compare labels on the remaining rows
keep = df["sold"] < df["sold"].quantile(0.99)
X_trim = StandardScaler().fit_transform(df.loc[keep, FEATURES])
km_trim = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(X_trim)
print("ARI full vs without top 1% sellers:", round(adjusted_rand_score(df.loc[keep, "kmeans"], km_trim), 3))

sensitivity = pd.DataFrame({
    "check": ["K-Means across 10 seeds (mean ARI)", "z-score vs min-max scaling (ARI)", "with vs without top 1% sellers (ARI)"],
    "ARI": [round(np.mean(seed_ari), 3),
            round(adjusted_rand_score(df["kmeans"], km_minmax), 3),
            round(adjusted_rand_score(df.loc[keep, "kmeans"], km_trim), 3)],
}).set_index("check")
save_table(sensitivity, "table12_sensitivity_checks.csv")
sensitivity
```

Figure 26. Code screenshot: stability across random subsamples Source file: COM7022 - Assignment/output/screenshots/code14_stability.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots/code14_stability.png

5. Cluster characteristics

Translated back into business units (Table 11, Figure 27 and Figure 28), the four K-Means clusters of the 997 listings are:

- **Cluster 2, Best-sellers (242 listings, 24.3%).** They deliver **92.3% of all units sold** (median 952 units) at a median price of $29.94, with full stock and 99.6% shipped from the US. Mostly Eau de Toilette; brands include Armaf and Ralph Lauren.
- **Cluster 0, Premium designer (219 listings, 22.0%).** The highest prices (median $68.99, up to $259.09), steady but slow sales (median 43 units, 3.7% of volume), deep stock. Mostly Eau de Parfum from Giorgio Armani, Paco Rabanne and Yves Saint Laurent.
- **Cluster 1, Budget (224 listings, 22.5%).** The lowest prices (median $18.99) but weak sales (median 30 units, 1.8% of volume). It holds the most unbranded and generic items, so low price alone does not create demand.
- **Cluster 3, Low-stock tail (312 listings, 31.3%).** The largest group: median stock of only 3 units, 14.1% marked “Last One”, median 16 units sold (2.2% of volume).

Measure	Cluster 0 Premium	Cluster 1 Budget	Cluster 2 Best-seller	Cluster 3 Low-stock
Listings	219	224	242	312
Share of listings %	22.0	22.5	24.3	31.3
Median price (US$)	68.99	18.99	29.94	39.99
Median units sold	43.0	30.0	952.0	16.0
Share of units sold %	3.7	1.8	92.3	2.2
Median stock level	11.0	11.0	11.0	3.0
% marked Last One	0.0	0.0	1.2	14.1
Top product type	Eau de Parfum (45.2%)	Eau de Toilette (37.9%)	Eau de Toilette (67.8%)	Eau de Toilette (39.1%)

Table 11. Profile of the four K-Means segments. Source file: COM7022 - Assignment/output/tables/table13_cluster_profiles.csv | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables/table13_cluster_profiles.csv

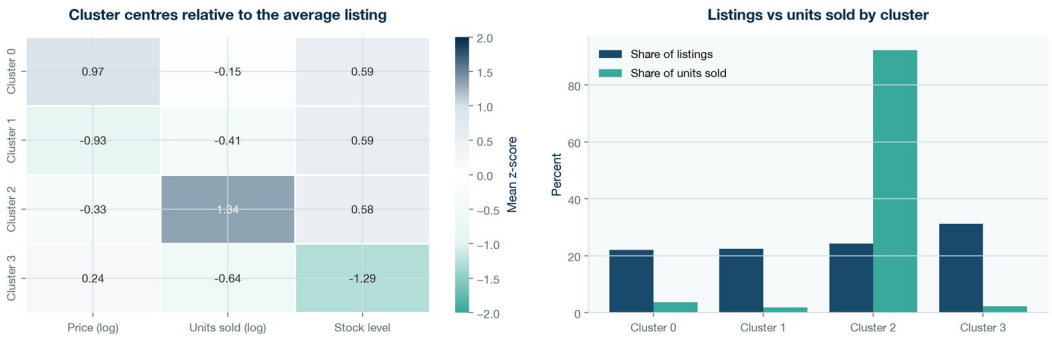


Figure 27. Cluster profiles as standardised feature means. Source file: COM7022 - Assignment/output/figures/fig13_cluster_profiles.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig13_cluster_profiles.png

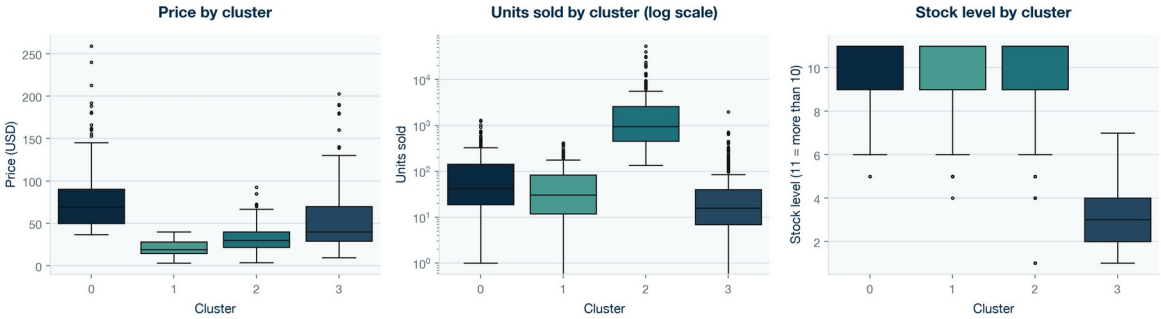


Figure 28. Price, units sold and stock distributions inside each cluster. Source file: COM7022 - Assignment/output/figures/fig14_cluster_distributions.png | https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures/fig14_cluster_distributions.png

Task 2: Critical Report for Executives

6. Executive summary

Our 997 fragrance listings fall into **four natural segments**. One quarter of them, the best-sellers, generate **92% of all units sold**. The other three segments (premium, budget and low-stock listings) together account for 8% of sales and need three different treatments. I recommend **K-Means with four clusters** as the company's segmentation model: it is the most accurate, the most stable and the simplest to run on new listings. The groups overlap somewhat, as is normal for real catalogues, so they should guide decisions, not replace judgement.

7. Why this model, and why for this dataset

7.1 Suitability of the methods and decisions

Unsupervised clustering suits the problem because the company has no labels. Three decisions mattered most. Log scaling stopped a handful of viral listings from controlling the result. Choosing price, sales and stock as features answered the three questions managers ask (what is it worth, does it sell, will it run out). Keeping brand and country out of the distance calculation prevented clusters from simply being brand lists.

7.2 How the models performed

K-Means led on all internal metrics and on stability (Table 9, Figure 24). Ward was moderately good but changed whenever the sample changed. DBSCAN found almost no structure: 90.7% of listings in one cluster and 8.9% noise. This is informative rather than a failure: the listings sit on a continuum from budget to premium and from slow to fast, so methods that assume gaps between groups cannot work well. A silhouette of about 0.31 confirms the groups are meaningful but not sharply separated.

Strengths and limits of each model. *K-Means* is fast, stable and explainable, but it assumes round groups of similar size and needs k chosen in advance; here a continuum makes the borders soft. *Ward* shows how groups nest and needs no k, but it is unstable and cannot score new listings. *DBSCAN* is ideal for finding outliers and irregular shapes, but this catalogue has no empty gaps between groups, so it merges most listings into one cluster. No model is perfect for this data; K-Means has the fewest weaknesses that matter to the business.

7.3 Which model is best for e-commerce?

Technical scores are only half the decision. A marketplace also needs a model that covers every listing, can be re-run every week, can score a new listing instantly and can be explained to a manager (Table 12). K-Means meets all of these. Ward cannot score new listings without re-building the tree. DBSCAN leaves listings unassigned, but it is useful as a monitor for unusual listings.

A fair question is why not simply rank listings by sales. A ranking shows who sells most, but not who is premium, who is cheap and slow, or who is about to run out. Clustering combines price, sales and stock in one view and yields groups that each need a different action. That is the difference between a league table and a segmentation.

E-commerce requirement	K-Means	Ward hierarchical	DBSCAN
Covers every listing	Yes (0% noise)	Yes (0% noise)	No (8.9% noise, 90.7% in one group)
Scores a new listing instantly	Yes: nearest centre	No: tree must be re-built	Partly: needs the full data
Stable week to week	Yes (ARI 0.93)	No (ARI 0.48)	Moderate (ARI 0.75)
Easy to explain to managers	Yes: four typical profiles	Medium: dendrogram	Hard: noise and one big group
Scales to millions of listings	Yes	Poor (memory grows with the	Medium

		square of listings)	
Best role	Main segmentation	Cross-check	Outlier alert

Table 12. Suitability of each model for an e-commerce catalogue.

8. Recommendations for managers

Each recommendation follows from the evidence in one segment (Table 11):

- **Protect the best-sellers (Cluster 2).** They carry 92% of sales. Set automatic restock alerts when stock falls below 10 units and give these sellers priority in search and fulfilment.
- **Grow the premium segment (Cluster 0).** Demand is steady but slow. Bundle these listings with best-sellers on product pages and invest in better images and descriptions rather than discounts.
- **Rethink the budget segment (Cluster 1).** Low prices do not create sales without brand recognition. Stop paid promotion of unbranded items and move that budget to proven brands.
- **Fix the low-stock tail (Cluster 3).** Ask sellers to restock within 30 days, and delist listings that stay empty and unsold.
- **Operate the model.** Re-run the segmentation monthly, assign new listings to the nearest centre, and review DBSCAN's noise points as unusual listings.

Start with the best-sellers and the low-stock tail, because they need the least effort and protect the most revenue. Measure success by stock-outs in Cluster 2, the share of Cluster 3 listings restocked or delisted within 30 days, and sales per advertising dollar in Cluster 1.

9. Limitations

The marketplace caps visible stock at 10, so stock above that is unknown. The data is one snapshot of one product family, so the segments may differ for other categories. The clusters describe association, not cause: a premium price does not by itself make a listing slow. Finally, the structure is a continuum, so listings near a boundary can fall in either group.

These limits do not change the main finding, but they set how the results should be used: as a guide for prioritising attention, reviewed monthly, with the stock cap and the narrow product range kept in mind.

10. Conclusion and future work

This project shows that a simple, well-prepared K-Means model can turn an unlabelled catalogue into four clear and actionable segments. My main lesson is that preparation mattered more than algorithm choice: recovering values from the listing text and handling the hidden stock cap changed the result more than any tuning. I also learned to treat a weak result honestly: DBSCAN's poor fit taught us something real about the data.

To extend the work I would add real stock counts and revenue per listing, include text features from the descriptions, track segments over time to see which listings move between them, and test whether the segments predict future sales. I would also build a small dashboard so managers can see each listing's segment.

References

Davies, D.L. and Bouldin, D.W. (1979) 'A cluster separation measure', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), pp. 224-227.

Caliński, T. and Harabasz, J. (1974) 'A dendrite method for cluster analysis', *Communications in Statistics*, 3(1), pp. 1-27.

Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996) 'A density-based algorithm for discovering clusters in large spatial databases with noise', in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. Portland: AAAI Press, pp. 226-231.

Hubert, L. and Arabie, P. (1985) 'Comparing partitions', *Journal of Classification*, 2(1), pp. 193-218.

MacQueen, J. (1967) 'Some methods for classification and analysis of multivariate observations', in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Berkeley: University of California Press, pp. 281-297.

Müller, A.C. and Guido, S. (2016) *Introduction to machine learning with Python*. Sebastopol: O'Reilly Media.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830.

Pearson, K. (1901) 'On lines and planes of closest fit to systems of points in space', *Philosophical Magazine*, 2(11), pp. 559-572.

Rousseeuw, P.J. (1987) 'Silhouettes: a graphical aid to the interpretation and validation of cluster analysis', *Journal of Computational and Applied Mathematics*, 20, pp. 53-65.

Ward, J.H. (1963) 'Hierarchical grouping to optimize an objective function', *Journal of the American Statistical Association*, 58(301), pp. 236-244.

Appendix A: Code, repository and reproducibility

The full Task 1 code is in one Jupyter notebook that runs end to end (about 30 seconds) on Google Colab or locally. All materials are hosted in the GitHub repository below.

Repository: https://github.com/kvmmn/_COM7022_MachineLearning

Notebook (full code): https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/COM7022_Clustering_25199053.ipynb

Data: https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/Sales-%5B5444%5D.csv

Figures: https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/figures

Tables: https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/tables

Code screenshots: https://github.com/kvmmn/_COM7022_MachineLearning/blob/main/COM7022%20-%20Assignment/output/screenshots

Interactive review page: https://kvmmn.github.io/_COM7022_MachineLearning/

Remaining code screenshots (clustering, hierarchical, DBSCAN and evaluation cells) appear in Sections 3 and 4. The notebook uses fixed random seeds (42), so every figure and table can be regenerated.

AI use declaration: AI tools were used to help structure and proofread this work. All analysis was run, checked and interpreted by me, and every number in this report was verified against the notebook output. (Edit to match the Arden AI policy.)